MechanicsDSL

Engine Reference and Baseline Freeze Record
for the Silent-Failure Study

Noah Parsons

Pin `a8dc2b2` (`v2.1.3`) · Frozen 22 August 2026

Abstract

This document is the reference record for the engine under measurement in the silent-failure study. It fixes the exact revision of MechanicsDSL that the study's numbers describe, enumerates every change between the previous pin `d04207c` and the current pin `a8dc2b2`, establishes which of those changes can and cannot reach the measured code paths, describes the engine's architecture and the subset of it the measurement exercises, and records the resulting baseline. Its purpose is falsifiability: a reader should be able to reconstruct the measured artefact exactly and check every number against it. Where a claim in this document rests on inference rather than observation, it is labelled as such.

Contents

1 How to read this document

The study this document supports asks one question:

> *Does a physics engine tell you when it is wrong, or does it hand you a wrong answer and claim it worked?*

A number answering that question is worth exactly as much as the provenance behind it. Two facts have to hold before any such number means anything: the engine measured must be identifiable to the commit, and the checks behind the verdict must have actually executed. Section 2 establishes the first. Section 5 establishes the second.

Three claim strengths are used throughout, and are always marked:

- **Observed** — read directly from the repository, the run artefacts, or the interpreter. Reproducible by the commands in Appendix A.
- **Derived** — follows from observed facts by stated reasoning. The reasoning is given so it can be attacked.
- **Expected** — a prediction not yet confirmed by measurement. Never load-bearing for a reported number.

2 The pin

2.1 What is frozen

Observed.

Field	Value
Commit	a8dc2b27ab1110d1f08dd24d0d6a6bdd994b4599
Short	a8dc2b2
Tag	v2.1.3 (annotated)
Authored	2026-08-15 11:44:07 −0600
Author	Noah Parsons
Subject	release: v2.1.3 - correctness release for ARM, Hamiltonian, and degenerate solves
Package	mechanicsdsl-core 2.1.3
Repository	https://github.com/MechanicsDSL/mechanicsdsl
Frozen on	2026-08-22

The freeze is recorded in the repository at `stress_suite/FREEZE.md`, which is the operative anchor; this document is its expanded form. Where the two disagree, `FREEZE.md` governs, because it is the file the sweep harness and the scope document both point at.

2.2 Provenance of the pin

Observed. The engine has been pinned twice.

Pin	Date	Status
d04207c	10 Aug 2026	Superseded. Baseline measured here is archived in `RESULTS.md` and marked pending re-verification.
a8dc2b2	22 Aug 2026	Current. Baseline in Section 6.

The governing scope document states that any engine change voids the baseline and requires a re-run before any number is cited again. Seven commits landed after `d04207c`. The pin was therefore moved deliberately and the sweep repeated, rather than allowing the recorded pin to drift away from the measured artefact. Section 3 is the accounting of what moved.

2.3 Reproducing the frozen engine

```
git clone https://github.com/MechanicsDSL/mechanicsdsl.git
cd mechanicsdsl
git checkout v2.1.3
```

Observed. At the time of freezing, the working tree matched `a8dc2b2` for all of `src/` and `pyproject.toml`. One untracked file, `RELEASE_NOTES_v2.1.3.md`, was present; it is release prose and contains no executable code, so it cannot affect the measurement.

2.4 Measurement environment

Observed.

Component	Version
Operating system	Windows 11 Home 10.0.26200
Python	3.14.0
`sympy`	1.14.0
`numpy`	2.3.5
MechanicsDSL	2.1.3, loaded from `src/`

2.5 A reproducibility hazard worth recording

Observed. The measurement host also has MechanicsDSL **2.0.6** installed into `site-packages`. A bare interpreter on this host resolves `import mechanics_dsl` to that stale copy:

```
$ python -c "import mechanics_dsl as m; print(m.__version__, m.__file__)"
2.0.6 C:\Python314\Lib\site-packages\mechanics_dsl\__init__.py
```

The sweep harness defeats this. `run.py` prepends the repository's `src/` directory to `PYTHONPATH` in the environment it hands to each worker subprocess, and `PYTHONPATH` precedes `site-packages` on `sys.path`. Under that environment the same import resolves correctly:

```
$ PYTHONPATH=.../mechanicsdsl-main/src python -c "..."
2.1.3 C:\Users\...\mechanicsdsl-main\src\mechanics_dsl\__init__.py
```

This was verified explicitly rather than assumed, because the failure it guards against is precisely the failure the study is about: the sweep would have measured 2.0.6, reported success, and labelled the result 2.1.3. It would have been a silent failure in the instrument rather than in the engine, and nothing in the output would have revealed it. Anyone reproducing this work on a host with MechanicsDSL installed should confirm the resolved version before trusting a result.

3 The delta: `d04207c` to `a8dc2b2`

3.1 Commits

Observed. Seven commits, all authored by Noah Parsons. `d0d954e` is the first commit *after* the old pin; `a8dc2b2` is the new pin. An eighth commit, `9552c7d`, records the pin move itself and touches only `stress_suite/` documentation.

Commit	Date	Subject
`d0d954e`	10 Aug	`stress_suite`: zero-silent-failure baseline at `d04207c`
`04a7a3b`	14 Aug	`fix(codegen)`: correct target count and repair CLI, REST, and Modelica export
`71173e4`	14 Aug	`fix(arm)`: generate embedded derivatives from the system's equations
`0e64268`	14 Aug	`fix(arm)`: parenthesise inlined integer powers
`9b46781`	15 Aug	`examples`: force UTF-8 stdout and drop BOMs

Commit	Date	Subject
7b7198c	15 Aug	docs: add the mathematical and physical formulation writeup
a8dc2b2	15 Aug	release: v2.1.3
9552c7d	22 Aug	stress_suite: move the engine freeze to a8dc2b2

3.2 Source changes

Observed. Restricted to src/, the delta is five files, 241 insertions against 77 deletions.

File	Lines	Change
codegen/arm.py	160	Embedded derivative generation; bare-metal C printer; integer-power parenthesisation
cli.py	48	Target count; dispatch routed through PhysicsCompiler.export()
integrations/modelica.py	56	Export repair
server/routes.py	52	REST export repair
__init__.py	2	__version__ 2.1.2 → 2.1.3

Outside src/: one test file (tests/unit/test_codegen_arm.py, +116 lines), pyproject.toml (version), CHANGELOG.md, CITATION.cff, README.md, the new docs/MATH_AND_PHYSICS.{tex,pdf}, and 43 example scripts re-encoded to UTF-8 without BOM.

3.3 Reachability: can the delta affect the measurement?

This is the load-bearing question of the whole freeze, so the reasoning is given in full.
Observed. The sweep reaches the engine through exactly one import, in stress_suite/worker.py:

```
from mechanics_dsl import PhysicsCompiler
```

The worker does not invoke the command-line interface, does not start the REST server, does not call any code generator, and does not touch the Modelica integration. It compiles a DSL source string and integrates the result.
Derived. Mapping the five changed files against that path:

File	Role	On the measured path?
codegen/arm.py	Bare-metal ARM C emission	No
cli.py	Console entry point	No
server/routes.py	HTTP surface	No
integrations/modelica.py	Modelica export	No
__init__.py	Version string only	Immaterial

No symbolic, solver, parser, compiler, or domain module changed between the two pins. The conclusion — that the sweep should reproduce the prior tally — is therefore **expected**, not established, and Section 6 reports what measurement actually returned. The prediction is stated here so that a divergence would be a finding about the instrument rather than a surprise absorbed into the result.

3.4 Clarifying the release title

A reader comparing the release title to the delta will find an apparent contradiction. The v2.1.3 release is titled *correctness release for ARM, Hamiltonian, and degenerate solves*, yet no Hamiltonian or solver file appears in the `d04207c` → `a8dc2b2` delta above. The resolution is that those two fixes predate the old pin.

Observed. The Hamiltonian and degeneracy fixes are carried by two commits between tag `v2.1.2` (`f4706e5`) and the old pin `d04207c`:

Commit	Subject
`b582e4f`	`fix`: make degenerate solves fail loudly and defer large mass-matrix inversion
`ac23ff9`	`fix(solver)`: scale integration tolerance to mass-matrix conditioning, fail on runtime degeneracy

`b582e4f` touches `compiler.py` (+182), `symbolic.py` (±160), and `solver/core.py` (+64). It contains the simultaneous momentum inversion described in the changelog: the Legendre transform previously solved each velocity from its own momentum relation independently, which is valid only for uncoupled kinetic terms, and now inverts the momentum system as a whole and raises on a non-invertible relation instead of returning a Hamiltonian assembled from partial substitutions.
Derived. Both commits are ancestors of `d04207c`, verified by `git merge-base --is-ancestor`. They were therefore already present in the engine when the 10 August baseline was measured. The release notes group them under 2.1.3 because 2.1.3 is the release that *ships* them, not because they landed in the delta. **The changelog describes a release; this document describes a diff.** They answer different questions and both are correct.

This distinction matters beyond bookkeeping. The prior baseline is sometimes read as measuring a pre-fix engine; it does not. It measured an engine that already contained the Hamiltonian and degeneracy corrections, which is part of why the tally it reported was as clean as it was.

4 MechanicsDSL

4.1 What it is

Observed. MechanicsDSL is a domain-specific language and transpiler for classical mechanics, distributed as `mechanicsdsl-core` under the MIT licence. A user writes a mechanical system in LaTeX-inspired notation — coordinates, parameters, and a Lagrangian or Hamiltonian — and the toolchain derives the equations of motion symbolically, integrates them numerically, and optionally emits standalone source for one of twelve code-generation targets.
The twelve targets are enumerated in `PhysicsCompiler._GENERATOR_REGISTRY`: `cpp`, `python`, `rust`, `julia`, `matlab`, `fortran`, `javascript`, `cuda`, `openmp`, `wasm`, `arduino`, and `arm`. They are *targets* rather than twelve distinct languages: `cuda` and `openmp` are C/C++ dialects, `arduino` is C++ for a specific platform, `arm` is bare-metal C, and `wasm` is a compilation target rather than a source language.

Field	Value
Distribution	`mechanicsdsl-core`
Version	2.1.3
Licence	MIT
Python	≥ 3.9
Core deps	`numpy`$\geq$1.20, `scipy`$\geq$1.7, `sympy`$\geq$1.8, `matplotlib`$\geq$3.4, `tqdm`$\geq$4.60
Maturity	Declared *Production/Stable*
Source size	53,500 lines across 137 Python modules
Test files	108

4.2 The pipeline

Observed. A compilation proceeds through five stages.

1. **Parse** (`parser/`, 2,108 lines) — tokenise the DSL source and build an AST. Entry points `tokenize` and `MechanicsParser`.
2. **Compile** (`compiler.py`, 1,460 lines) — the orchestrator. `PhysicsCompiler` is the sole public entry point used by the measurement.
3. **Derive** (`symbolic.py`, 931 lines, plus `domains/classical/`) — form the Euler–Lagrange or Hamilton equations and solve for accelerations. The mass matrix is assembled and the acceleration system solved *simultaneously*, not coordinate-by-coordinate.
4. **Integrate** (`solver/`, 2,273 lines across `core`, `symplectic`, `variational`) — numerical integration, with optional Numba-JIT and JAX backends.
5. **Emit** (`codegen/`, twelve targets) — optional export to C++, Python, Rust, Julia, MATLAB, Fortran, JavaScript, CUDA, OpenMP, WASM, Arduino, and bare-metal ARM.

Only stages 1–4 are exercised by this study. Stage 5 is where the delta's largest change lives, and it is not measured here.

4.3 Subsystem inventory

Observed. Largest modules, by line count.

Module	Lines	Role
compiler.py	1460	Pipeline orchestration
domains/quantum/core.py	1262	Quantum domain
domains/kinematics/projectile.py	1140	Projectile kinematics
solver/core.py	1089	Numerical integration
domains/solid_mechanics/beam_theory.py	1082	Beam theory
domains/solid_mechanics/elasticity.py	1052	Elasticity
domains/electromagnetic/core.py	971	Electromagnetism
symbolic.py	931	Symbolic derivation
parser/core.py	891	Tokeniser and parser
parser/ast_nodes.py	857	AST
codegen/cpp.py	833	C++ emission
codegen/arm.py	761	Bare-metal ARM emission
solver/symplectic.py	722	Symplectic integrators

The breadth is worth noting honestly: the package spans quantum, relativistic, general-relativistic, electromagnetic, thermodynamic, statistical, fluid, and solid-mechanics domains in addition to classical mechanics. The study measures the classical Lagrangian, Hamiltonian, and constrained pathways only. **No claim in the study extends to the other domains**, and the inventory is given here so that the scope limit is visible rather than implied.

4.4 Surrounding infrastructure

Observed. Beyond the pipeline, the package ships a REPL, a Language Server Protocol implementation, a FastAPI server with WebSocket support, Jupyter magics and display hooks, a plugin registry and loader, inverse-problem tooling (parameter estimation, sensitivity, uncertainty), visualisation and phase-space plotting, a units system, and integrations for ROS2, Unity, Unreal, OpenMDAO, and Modelica. None of these are on the measured path.

4.5 Test suite

Observed. 108 test files.

Directory	Files
tests/unit	48
tests/physics	37
tests/integration	6
tests/ (top level)	5
tests/property	3
tests/validation	3
tests/performance	2
tests/security	2
tests/edge_cases	1
tests/test_backends	1

The existence of this suite is not evidence against silent failure. Every defect corrected in `b582e4f` and `71173e4` was present while the suite passed. That is the study's premise rather than a criticism of the suite: a test asserts what its author thought to assert, and a silent failure is by construction the case nobody thought to assert.

5 The measuring instrument

5.1 Design

Observed. `run.py` generates a specification file per case, executes `worker.py` as a *subprocess* under a wall-clock timeout, and classifies the outcome. Subprocess isolation is deliberate: a segmentation fault or a hang in the engine must be recorded as a data point, not take down the harness.

5.2 Scoring

Observed. Four outcomes.

Box	Meaning
pass	Correct answer.
silent	*Wrong answer reported as success.* The quantity of interest.
error	Wrong or impossible, and said so. Acceptable behaviour.
timeout	Never returned. A missing answer, not a wrong one.

Timeouts are deliberately *not* folded into `error`. An errored case is one the engine adjudicated; a timed-out case is one it never answered. Folding them together would make the reported failure rate a function of the chosen timeout, which is a property of the harness rather than of the engine. The denominator for the silent-failure rate is therefore pass + silent + error, and the timeout count is reported separately as a scaling-wall result.

5.3 Oracles

Observed. A verdict rests on independent checks, not on the engine's own report.

Check	Method	Tolerance
Equations of motion	Engine's ODE right-hand side compared against an independent ground-truth derivation at $K = 12$ randomised states	10^{-6} rel.
Energy conservation	Relative drift over the trajectory	10^{-2}
Constraint residual	Absolute growth in the constraint residual	10^{-2}
Structural	NaN/Inf trajectories, all-zero equations of motion, frozen-when-displaced, and solve-fallback warnings riding on `success=True`	—

The right-hand-side probe is the strongest of these, and its introduction in `d04207c` was itself prompted by a defect in an earlier version of this harness, which reported passes on cases where the strongest check had quietly failed to execute. The harness now surfaces unverified passes explicitly. **A silent-failure rate is worth exactly as much as the checks behind it**, so the count of cases where the oracle actually ran is reported alongside the headline number in Section 6.

5.4 Oracle coverage is uneven, and this bounds the claim

The strong oracle covers under half the adjudicated cases. It applies to the unconstrained Lagrangian pathway only. Hamiltonian and constrained cases are adjudicated by energy conservation, constraint residual, and the structural checks — all weaker instruments.

Observed. Coverage in the prior baseline run, computed directly from `out/results.json`:

Pathway	Adjudicated	Strong oracle ran
`lagrangian`	23	22
`hamiltonian`	14	0
`constrained`	8	0
Total	**45**	**22**

Two separate facts live in that table, and they pull in opposite directions.

The reassuring one: the oracle ran on *every* case where it was applicable — 22 applicable, 22 executed. This is the specific failure the harness was rebuilt to prevent, and the rebuild worked. It is what licenses the statement in `RESULTS.md` that the check actually ran.

The limiting one: **23 of 45 adjudicated cases carry no independent derivation check at all.** A headline of "zero silent failures in 45 cases" is, more precisely, zero silent failures in 22 cases checked against an independent derivation plus 23 cases checked only for energy drift, constraint drift, and structural pathology.

This bound deserves emphasis because of where it falls. The most serious silent failure in the engine's history — the two-link Hamiltonian that compiled, reported success, and sat motionless — was on the pathway that receives *zero* strong-oracle coverage. That particular defect is caught by the frozen-when-displaced structural check, so it is not invisible to the harness. But a Hamiltonian system that moves plausibly while solving the wrong equations would pass energy conservation and every structural check, and nothing in the current suite would contradict it.

Derived. The claim this evidence supports is therefore pathway-dependent, and should be stated that way rather than pooled:

- **Lagrangian:** no silent failures, verified against an independent derivation.
- **Hamiltonian and constrained:** no silent failures detected by energy, constraint, and structural checks. Weaker evidence, and it should not be pooled with the above into a single number without the qualifier.

5.5 Where the gaps fall, jointly

Observed. The oracle gap and the timeout wall are usually reported separately. They should not be, because they fall in the same place and compound.

Pathway	Cases	Timed out	Adjudicated	Strong oracle
`lagrangian`	26	3	23	22
`hamiltonian`	19	5	14	0
`constrained`	10	2	8	0
Total	**55**	**10**	**45**	**22**

Derived. Thirty-three of the 55 cases — 60 % of the matrix — carry no independent derivation check. Of those, seven produced no answer at all. Non-Lagrangian evidence therefore amounts to 22 cases adjudicated at the weak tier, with the region past three Hamiltonian links and four loop nodes entirely dark.

Stated separately, the two caveats sound like minor qualifications. Stated jointly, they define the actual shape of the evidence: one pathway is well covered, and the other two are thinly covered and go blind exactly where the systems get hard. This is the honest bound, and it belongs in the paper in this form.

5.6 The remedy, and why it is affordable

A single artefact closes both gaps: **a library-independent reference implementation of the planar N-link chain.** The planar chain has a closed-form mass matrix, Coriolis term, and gravity vector,

$$M(q)\,\ddot{q} + C(q,\dot{q})\,\dot{q} + g(q) = 0, \tag{1}$$

requiring no symbolic algebra — a few hundred lines of `numpy` with a hand-checkable $N = 2$ case.

Such a reference shares no library with any of the three engines. That matters most for the SymPy column: a hand-rolled SymPy derivation checked against SymPy's `LagrangesMethod` exercises different code paths but shares `diff`, `simplify`, `solve`, and the assumptions machinery beneath them. Agreement under those conditions is close to a self-consistency test, and asymmetric evidence across the three columns is the one defect a comparison table cannot survive.

The same artefact extends strong-oracle adjudication to the Hamiltonian pathway on the portable family, and doubles as the Week 1 hand-checked gate case — which has to be written regardless. The marginal cost is generalising that gate case in N.

5.7 Case matrix

Observed. Six axes, three formulation pathways (`lagrangian`, `hamiltonian`, `constrained`), 55 (system, tool) cases.

Axis	Knob values	Tools	Cases
`dof`	$N = 1, 2, 3, 4, 5$	lag, ham	10
`loops`	$N = 3, 4, 5$	constrained	3
`redundancy`	$R = 0, 1, 2, 3, 4, 6, 8$	constrained	7
`near_singular`	$\varepsilon = 10^{-1} \ldots 10^{-11}, 0$	lag	7
`mass_ratio`	$10^{0} \ldots 10^{16}$	lag, ham	14
`symbolic`	depth $= 1, 2, 4, 8, 12, 16, 24$	lag, ham	14
		Total	**55**

Observed. The skip list (`KNOWN_SLOW`) is empty by deliberate choice. It was previously populated on the belief that high-DOF Hamiltonian cases hang past 600 s; the belief did not survive the data, as `dof_N4` and `dof_N5` on the Lagrangian side went from timing out at 180 s to finishing in under 12 s once the mass-matrix path changed. Every case is measured.

Scope note. Portability across engines splits the six axes as follows.

Axis	Cross-engine status
`dof`	Portable. Every engine expresses an N-link chain.
`near_singular`	Portable. A near-zero inertia is a number in a model file.
`mass_ratio`	Portable. Likewise.
`loops`	**Undecided.** Closed kinematic chains are well supported by Drake and expressible in SymPy via Lagrange multipliers, so this axis is arguably portable. It is not yet committed to the cross-engine claim.
`symbolic`	Not portable. Nesting depth in a Lagrangian expression is only meaningful to an engine that consumes raw Lagrangians.
`redundancy`	Not portable. Hand-built redundant constraint sets, same reasoning.

An earlier version of this section was wrong, and the error is recorded rather than quietly corrected. It listed four portable axes including "degenerate starting positions," which is not one of the six axes at all — it is one of the four *knobs* named in `SCOPE.md`. The two taxonomies are different: `SCOPE.md` describes what is dialled, the suite describes how cases are grouped. Conflating them also caused `loops` to be omitted from both lists.

The `loops` decision is load-bearing and must be made before the adapters are written. If the constrained pathway enters the cross-engine claim, three more cases join a pathway that currently has zero strong-oracle coverage (Section 5.4). If it does not, the claim covers unconstrained dynamics only and should say so.

6 Baseline

6.1 Result

Observed. Sweep executed 22 August 2026 at pin `a8dc2b2`, 180 s per-case wall-clock timeout, 55 cases.

Outcome	Count	Share of adjudicated
pass	44	97.8 %
silent	**0**	**0.0 %**
error	1	2.2 %
Adjudicated total: **45**		
timeout	10	— (reported separately)
Total cases	**55**	

The baseline: zero silent failures in 45 adjudicated cases, at pin `a8dc2b2` (`v2.1.3`), with the independent ground-truth oracle executing on all 22 cases where it was applicable. Ten cases did not finish within 180 s and are excluded from the denominator as missing answers rather than wrong ones.

6.2 Reproduction against the prior pin

Observed. The prediction in Section 3.3 was that the sweep would reproduce, because no module on the measured path changed between the pins. It did, and more exactly than anticipated.

Quantity	`d04207c`	`a8dc2b2`
pass / silent / error / timeout	44 / 0 / 1 / 10	44 / 0 / 1 / 10
Adjudicated cases	45	45
Oracle applicable / ran	22 / 22	22 / 22
Per-case status changes	**0 of 55**	
Max change, any recorded metric	0.0 **(exact)**	

Not one case changed status, and every recorded numeric quantity — energy drift, equation-of-motion mismatch, constraint residual, position range — is identical to the last bit. The engine is deterministic across runs on this host, and the `d04207c→a8dc2b2` delta is confirmed inert with respect to the measured path.

Derived. Two consequences follow.

First, the `d04207c` baseline and the `a8dc2b2` baseline are the same measurement, not merely compatible ones. The reasoning in Section 3.3 is now supported by observation rather than standing alone, and the numbers archived in `RESULTS.md` may be cited at the current pin.

Second — and this is the more useful consequence — exact reproduction **forecloses an entire class of objection to the cross-engine comparison before it can be raised.** The natural challenge to any Week 2 result is that observed differences between engines might be run-to-run noise in the harness rather than genuine disagreement. That challenge is now answerable with a null result measured on exactly the right question: 55 cases, two independent invocations twelve days apart, zero variation in any recorded metric. There is no unseeded randomness in the oracle's probe states and no timing-dependent branch, so any difference the comparison later shows is attributable to the engines.

This belongs in the paper, not only in this record. A reviewer who asks "how do you know that isn't noise?" should be met with a measurement rather than an argument.

It also supplies something the study needs operationally: a **bit-exact regression test on the instrument itself**. Any future extension of the harness can be validated by re-running the frozen case matrix and confirming it still returns 44/0/1/10 with identical metrics. See Section 7.

Interpretation. A zero here means *no silent failure was observed in this case set*, not that the engine does not produce them. See Section 5.4 on which cases carry an independent derivation check, and Section 8, item 3, for the statistical bound.

6.3 Prior baseline, for comparison

Observed. The 10 August sweep at pin `d04207c` reported **zero silent failures in 45 adjudicated cases**: of 55 cases, 44 passed, 1 was an explicit refusal, and 10 did not finish. The single refusal was `nearsing_e0`, an exactly singular mass matrix — a genuinely degenerate system in which the coordinates cease to be independent and there is no motion to compute. The engine fails at compile time and says so, which is correct behaviour and is scored `error`, not `silent`.

7 Governance: what is frozen, and what may grow

An earlier statement of the freeze rule was too broad and would have blocked the study's own next step. It read: *the instrument does not change either; improvements to the harness are recorded, not applied.* Week 1 consists of two new engine adapters and a multi-engine dispatch layer — all of it harness work. The rule and the plan contradicted each other.

The rule was over-tightened by analogy: because the engine must not change mid-measurement, it seemed to follow that nothing may change. That does not follow. The reason for freezing the engine is that a moving engine makes the numbers describe no particular artefact. The harness has a different property — it can be extended and then *proved* not to have disturbed what came before.

7.1 The rule, correctly scoped

Frozen	**Why**
The engine at `a8dc2b2`	A moving engine makes the numbers describe nothing in particular.
The MechanicsDSL scoring path	The classifier that produced 44/0/1/10 must not be re-tuned after seeing results.
The case matrix (six axes, 55 cases)	Adding or dropping cases after the fact is selection on the outcome.
May grow	**Condition**
New engine adapters (SymPy, Drake)	Additive. Cannot alter the MechanicsDSL column.
Multi-engine dispatch	Same.
New oracles (e.g. the N-link reference of Section 5.6)	May *add* adjudication where there was none; may not re-adjudicate an existing verdict.

7.2 The re-validation gate

Every extension of the harness must satisfy one condition before it is trusted:

> Re-run the frozen case matrix at `a8dc2b2`. It must still return **44 pass / 0 silent / 1 error / 10 timeout**, with every recorded metric bit-identical to the committed baseline.

This is enforceable rather than aspirational precisely because reproduction was shown to be exact (Section 6). A harness change that perturbs the MechanicsDSL column announces itself immediately. One that does not is additive by demonstration.

The distinction that matters is between extending coverage and revising judgement. Adding an independent check to a pathway that had none is extension: it can only turn an unverified pass into a verified pass or a detected failure. Loosening a tolerance, reclassifying a warning, or dropping an inconvenient case after seeing its result is revision, and remains forbidden for the duration.

8 Threats to validity

1. **The author maintains one of the engines measured.** MechanicsDSL is the author's own project. This is disclosed here and must be disclosed in any publication drawing on these numbers. A maintainer measuring their own tool is normal and defensible when stated, and indefensible when

concealed. The concrete mitigation is that verdicts rest on an independent ground-truth oracle and on cross-engine disagreement rather than on the engine's self-report.

2. **Single-engine results do not support the cross-engine claim.** The claim the study defends — that engines disagree on the same system and some do not warn you — requires the other two engines. This document records one engine's baseline only. Nothing here is evidence for or against the comparative claim.

3. **A zero is a weak observation.** Zero silent failures over 45 cases bounds the rate loosely, not tightly. With 45 adjudicated trials and no events, the one-sided 95 % upper bound on the underlying rate is roughly $1 - 0.05^{1/45} \approx 6.4\,\%$. The honest statement is that no silent failure was observed in this case set, not that the engine does not produce them.

4. **The case set is adversarial, not representative.** Axes were chosen because they are known ways to break solvers. Rates measured here do not transfer to typical use.

5. **Timeout choice shifts the denominator.** At 180 s per case, 10 cases did not finish under the prior pin. A shorter timeout moves cases from the adjudicated pool into the timeout pool and changes the denominator. The timeout is therefore reported with every tally.

6. **The oracle shares a symbolic library with the engine, and this becomes acute for the SymPy column.** Both the ground-truth derivation and MechanicsDSL depend on `sympy`; a defect in `sympy` could affect both and escape detection. For MechanicsDSL this is a modest concern, mitigated by energy conservation, which requires agreement on the physics rather than on the derivation.

   It is not a modest concern for the second engine. When the engine under test *is* `sympy.physics.mechanics`, the strong oracle becomes a hand-rolled SymPy derivation checked against SymPy's `LagrangesMethod`. These are different code paths, so it is not strictly circular — but they share `diff`, `simplify`, `solve`, and the assumptions machinery beneath them, which makes agreement closer to a self-consistency test than to independent confirmation.

   The consequence is asymmetric evidence across the three columns: the MechanicsDSL column adjudicated independently on the Lagrangian pathway, the SymPy column adjudicated by something notably weaker on the same pathway, reported side by side under one heading. A comparison table does not survive that. The library-independent reference of Section 5.6 is the remedy, and it is a Week 1 dependency rather than a later improvement.

7. **Cross-engine energy drift may measure the integrator rather than the engine.** Drake's default integrator and error control are not `scipy`'s. On the Hamiltonian and constrained pathways, where drift is the only real oracle, comparing drift across engines risks attributing an integrator choice to engine correctness. Either pin the integrator family and tolerance across all three engines, or restrict cross-engine claims to the pathway where the ground-truth probe adjudicates. This must be decided in Week 1; discovering it in Week 2 is far more expensive.

8. **AI assistance was used** in preparing this analysis and document. It is permitted in the venues targeted, and is disclosed.

A Reproduction

Every **observed** claim in this document is reproducible with the following.

The pin and the delta:

```
git rev-parse v2.1.3^{commit}
git log --oneline d04207c..a8dc2b2
git diff --stat d04207c..a8dc2b2 -- src/
git merge-base --is-ancestor b582e4f d04207c && echo "predates old pin"
```

The environment:

```
python -c "import sys, sympy, numpy; print(sys.version, sympy.__version__, numpy.
    __version__)"

# Resolved engine version. The measurement host is Windows, so both forms:
# PowerShell
$env:PYTHONPATH = ".\src"; python -c "import mechanics_dsl as m; print(m.__version__,
     m.__file__)"
# POSIX shell (Git Bash, WSL, Linux, macOS)
PYTHONPATH=./src python -c "import mechanics_dsl as m; print(m.__version__, m.
    __file__)"
```

The sweep:

```
cd stress_suite
python run.py --timeout 180
```

Artefacts are written to `stress_suite/out/`: `results.json` (per-case records) and `report.md` (tally). The report may be regenerated from existing records without re-running any case:

```
python run.py --report-only
```

Governing documents in the repository:

File	Authority
`stress_suite/SCOPE.md`	The study plan. Authoritative on scope.
`stress_suite/FREEZE.md`	The engine pin. Authoritative on what was measured.
`stress_suite/RESULTS.md`	The baseline tally.
`stress_suite/FIXES.md`	Case-by-case account of pre- and post-fix differences.
`stress_suite/MASTER_REPORT.md`	Historical, 28 July. Describes a pre-fix engine and *must not* be quoted as current.
`docs/MATH_AND_PHYSICS.tex`	The engine's mathematical formulation.